

面向个人电脑环境的轻量化 LLM 软件缺陷修复方法研究

——基于 HapRepair 框架的实证分析

方向 3-小组 3

成员:

组长: 高敏耀 23009200766

其他组员:

胡锦阳 23009200759

刘峻希 23009201434

李许涵 23009200590

方明远 23009200803

摘要

软件缺陷检测与修复是保障软件质量的关键环节。大型语言模型在该领域展现出巨大潜力，但高昂的计算成本限制了其在个人电脑上的部署。本研究聚焦于 HapRepair 框架，通过缩小 LLM 参数量、优化检索增强生成策略及扩展修复数据集，探索其在本地硬件环境下的可行性。实验选取了四款小参数量模型（GPT-OSS-20B, Qwen3-Coder-30B, Deepseek-R1-8B, Gemma 4 26B），在五个 OpenHarmony 项目上进行评估。结果表明，采用 MoE 架构的 Qwen3-Coder-30B 在 top_k=1 设置下表现最优，能够有效修复大部分缺陷，而 Deepseek-R1-8B 等模型因推理不稳定导致不可用。本研究证实，通过合理的模型选择和参数调优，可以在个人电脑上实现接近高端服务器的缺陷修复能力，显著降低了技术门槛和应用成本。然而，研究也揭示了小模型对高质量 RAG 知识库的高度依赖以及模型选择的重要性。

1. 引言

软件缺陷可能导致一系列严重后果，如系统崩溃、功能故障和安全漏洞，这些都会显著影响用户体验。此外，缺陷的累积增加了软件维护的复杂性和成本。因此，在早期阶段检测并修复缺陷至关重要，以确保软件的质量和长期可持续性^[12]。

现有的基于学习的方法通常在大量数据上训练模型。近年来深度学习的进展，尤其是基于 transformer 的模型，展现出从现有数据集学习模式和在评估基准测试中取得高性能的显著

能力提出缺陷修复建议。^{[2][3]}然而，ArkTS 是一门缺乏公开数据的新兴的编程语言，因为缺乏训练样本，这将直接影响 LLM 对该语言的代码在有效缺陷检测和修复方面的表现。

此外，LLM 本身固有的限制也构成了实际应用的瓶颈。首先，上下文窗口的最大长度限制了模型一次性处理的代码量，在处理包含多个缺陷的大型项目时尤为突出。其次，运行大规模 LLM 所需的算力和显存资源极其昂贵，通常需要依赖高端服务器集群或云服务，这极大地阻碍了技术在个人开发者和小型团队中的普及。^[5]

HapRepair 这一种新颖的方法旨在检测和修复 OpenHarmony 应用中的代码缺陷。该方法集成了静态分析工具 CodeLinter 进行缺陷检测；利用 RAG 通过上下文学习检索特定知识，通过手动构建修复范例来缓解训练数据不足的问题；环绕上下文提取器和上下文组合工具克服了多缺陷修复中上下文窗口的限制，通过仅保留修复任务的关键信息节省资源。

HapRepair 结合了静态分析的精确缺陷检测能力与 LLMs 的生成修复优势。^[1]

然而，HapRepair 的原始实现依赖于参数量巨大的 LLM（如 72B 及以上），这使得其在个人电脑上几乎无法运行。为解决此问题，本研究以 HapRepair 框架为基础，系统性地探究了将 LLM 轻量化以适配个人电脑环境的可能性。我们的核心研究问题是：通过缩小模型规模、优化 RAG 策略和扩充高质量修复数据集，能否在牺牲少量修复精度的前提下，实现本地化、低成本的缺陷自动修复？

本研究的主要贡献如下：

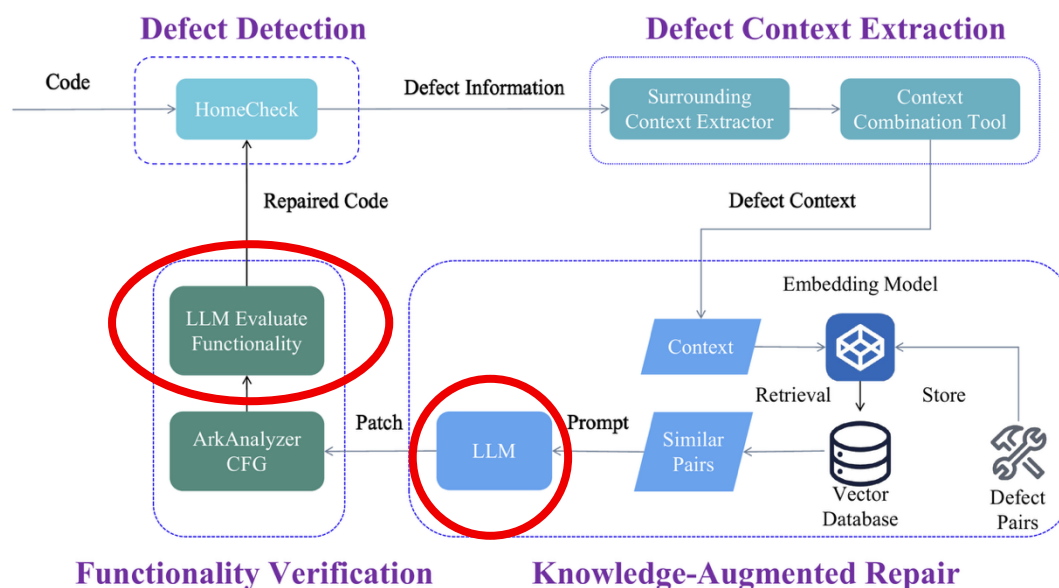
验证了小参数模型的可行性：通过实验验证了在个人电脑（16GB 显存 GPU）上，使用轻量化 LLM 进行软件缺陷修复的可行性，为本地化部署提供了实证依据。

扩展了高质量的 RAG 数据集：在原 HapRepair 提供的 271 对缺陷-修复样本基础上，手动构建并验证了额外的 365 对高质量修复范例，将总数据集扩展至 636 对，丰富了小模型的检索知识库。^{[6][7][9]}

改进了代码完整性检查机制：在 LLM 生成修复补丁后，增加了功能一致性验证步骤。若修复后的代码在关键行为（如编译错误、逻辑变更）上与原始代码不符，则自动回滚修改，有效避免了引入新缺陷的风险。

揭示了小模型部署的关键因素：通过对比实验，揭示了模型架构、领域相关性以及 RAG 检索深度（top_k 值）对小模型性能的决定性影响。

2. 框架分析、问题与改进方向



HapRepair 的流水线主要由以下几个模块构成（如上图）：

缺陷检测：使用静态分析工具 CodeLinter 扫描代码，识别潜在缺陷及其位置。

上下文提取：利用“环绕上下文提取器”获取缺陷周围的代码片段，并将其与项目级上下文结合。

RAG 检索：从构建的缺陷-修复范例库中，检索与当前缺陷最相似的 k 个修复对 (top_k)。

LLM 修复生成：将缺陷代码、上下文信息和检索到的相似范例拼接成提示词，输入 LLM，由模型生成修复补丁。

代码完整性检查：验证生成的补丁是否满足基本语法和功能一致性要求。其中开销最大的模块是两个 LLM 处理的部分（如上图标识，将代码上下文和相似的代码对传入 LLM 时需要大量计算，同时对修改后的代码进行功能性评估时还要经历一轮运算），

原始 HapRepair 方案中开销最大的模块是两次 LLM 调用：一次用于生成修复补丁，另一次用于评估修复后的代码功能。该结构虽然在理论上可行，但在实际应用中存在两大核心问题：

1. 高昂的硬件成本：原始方案运行在配备 4×NVIDIA A100 80GB GPU 的高端服务器上，使用 72B 甚至更大参数的 LLM。这对于个人电脑而言，无论是计算能力还是显存容量都无法满足，导致技术门槛极高。
2. 对云端服务的依赖风险：若转而使用第三方 API 进行 LLM 调用，虽能规避本地硬件问题，但会引入新的风险，包括：高度依赖网络服务导致的服务中断风险、按 Token 计费的持续高昂成本，以及在传输敏感代码时潜在的隐私泄露问题。

本研究正是针对上述局限性，致力于探索一条低成本、本地化的替代路径

本研究针对上述问题，以 HapRepair 框架为基础，系统性地探究了将 LLM 轻量化以适配个人电脑环境的可能性。我们的目标是验证：通过缩小模型规模、优化 RAG 策略和扩充高质量修复数据集，能否在牺牲少量修复精度的前提下，实现本地化、低成本的缺陷自动修复。

为了实现在个人电脑上运行 HapRepair 的目标，我们提出了以下三点核心改进：

1. 模型轻量化选择：用参数量适中、能在 16GB 显存 GPU 上运行的小参数 LLM 替代原始的大模型。我们选取了涵盖 MoE（混合专家）和 Dense（密集）两种架构的四款模型，并通过量化技术进一步降低内存占用。
2. RAG 数据集扩展：原始 HapRepair 提供的 271 对缺陷-修复样本对于小模型的知识储备而言可能不足。我们通过深入分析项目代码库和社区提交记录，手动构建并验证了额外的 365 对高质量修复范例，将总数据集扩展至 636 对，旨在为小模型提供更丰富的参考数据。
3. 代码完整性检查改进：在 LLM 生成修复补丁后，增加一个自动化的功能一致性验证步骤。该步骤会尝试编译修复后的代码，并进行关键行为的逻辑比对。一旦发现修复引入了编译错误或改变了程序的核心行为，系统将自动回滚至原始代码，从而避免引入新的问题。

3.实验设计

该实验选取了 4 个小参数量化版模型，在 RAG 检索深度为 1 和 5 时进行两轮实验：

GPT-OSS-20B（MoE 架构，mxfp4）；

Qwen3-Coder-30B（MoE，实际激活参数 3B，Q3_K_L）

Deepseek-R1-8B (dense, Q4_K_M)

Gemma 4 26B A4B QAT (MoE, 实际激活 4B, Q4_K_L)

参与性能评测的为原始论文出现过的 5 个项目：

Amqpplib; Arkts_tutorial; wifi_testapp; node_pop3; Gallery

选取的测试平台：

代码托管与运行（客户端）：

Apple Silicon M4 (10CPU+10GPU) , 24GB RAM, macOS 26.5.1

LLM 推理计算（服务端）：

Intel Core i9-13980HX (24C 32T) , 64GB DDR5 RAM, NVIDIA Geforce RTX 4090 Laptop GPU (16GB VRAM) , Windows 11 25H2, 使用 LM Studio 内置服务器进行推理, 服务器运行在本地网络上

对照（原始论文的配置环境, 非实际参与测试对象）：

Intel(R) Xeon(R) Platinum 8352V (144 cores) , 256GB RAM, 4 * NVIDIA A100 PCIe 80GB GPU, Ubuntu 24.04.1 LTS

控制变量：同样的虚拟环境、同一设备环境、相同初始状态, 固定采样参数

评估指标：

1. 模型经过一轮修复后剩下的缺陷数量（模型修复的缺陷个数）
2. 模型成功修复的缺陷比例
3. 在 top_k=1 和 top_k=5 两种情况下模型性能的区别

4.实验结果

下表展示了各模型在不同 top_k 设置下，对五个项目进行一轮修复后的剩余缺陷数。

“Err”表示模型在此配置下无法完成有效修复（如陷入死循环、输出无效代码）。

Model	GPT-OSS-20B		Qwen3-Coder-30B		Deepseek-R1-8B		Gemma 4 26B A4B QAT		Original defects
Project	top_k=1	top_k=5	top_k=1	top_k=5	top_k=1	top_k=5	top_k=1	top_k=5	
amqplib	17	6	6	6	13	Err	6	6	19
Arkts_tutorial	24	23	17	22	27	Err	Err	21	33
Gallery	17	11	17	18	Err	Err	Err	Err	25
node-pop3	34	13	8	8	Err	Err	Err	Err	73
wifi_test_app	26	20	3	11	Err	Err	Err	Err	109

5.结果分析

上述参与测试的 4 个模型均可以在 16GB 显存的电脑上独立运行，但是根据模型选择的不同，效果差异很大：

Deepseek-R1-8B 和 Gemma 4 26B 在多个项目中出现错误，主要表现为推理陷入无限循环或生成不符合格式要求的代码甚至直接超出 context window 报错，即使在调整 temperature 等参数后仍未改善。这表明并非所有小模型都适合此任务，模型架构和训练数据的领域相关性至关重要。

GPT-OSS-20B 和 Qwen3-Coder-30B 使用相同的 MoE 架构，运行开销较小，实测在测试环境并发数设置为 2 的情况下，在 RTX 4090 Laptop GPU 上可以达到 140 token/s 的输出速度。

其中表现最优的是 Qwen3-Coder-30B 在 top_k=1 的设置，在选取的 4 个模型内表现总体领先。在所有测试模型中，Qwen3-Coder-30B 在 top_k=1 的设置下表现最优。它在 wifi_testapp（109 个原始缺陷）上将剩余缺陷降至 3 个，在 node_pop3（73 个原始缺陷）上降至 8 个，修复效果显著。这归功于其 MoE 架构带来的高效计算，以及针对代码场景的微调所赋予的强大代码理解与泛化能力^[10]。

和原始论文数据存在差异的部分是：原始论文提出在 top_k 设置为 1 时表现最好，这是因为原始论文使用的模型都是 72B 或者更大的参量，内部数据规模足够大，而 GPT-OSS-20B 这种模型因为规模较小且未经过微调（Qwen3-Coder-30B 为代码场景微调的模型^[10]），需要更多的有效数据对才能作出有效的判断，故其在 top_k=5 时表现更好。这符合预期：小模型内部知识有限，需要更多外部相似案例（即更大的检索深度）来辅助决策。而对于 Qwen3-Coder-30B，top_k=1 反而略优于 top_k=5，这可能归因于其更强的代码理解和泛化能力，过多的噪声信息反而会干扰其判断。

原始 HapRepair 在高端服务器上对所有项目实现了接近零缺陷的修复。我们的轻量化方案虽然未能完全消除所有缺陷，但在多个项目上取得了显著成效（如 amqplib, node_pop3），证明了在可接受的性能折损下实现本地化部署是可行的。

研究暴露出来的局限性：

数据集的依赖性：本实验的成功高度依赖于手动扩展的 636 对修复数据集。对于全新的项目或语言，用户仍需投入精力构建高质量的 RAG 知识库。未来可探索自动化或半自动化的数据增强方法。

模型选择的敏感性：实验表明，模型的选择对最终效果有决定性影响。建议用户在部署前，在自己的目标项目上对候选模型进行小规模基准测试。

Top-k 策略：我们观察到，理想 top_k 值与模型容量和任务复杂度相关，应该根据实际情况动态调整

评估指标的局限性：当前仅统计了“剩余缺陷数”，未衡量修复代码的风格一致性等。未来应引入更全面的评估体系。

6.结论与展望

本研究通过实验验证了在个人电脑环境下，使用轻量化 LLM 进行软件缺陷修复的可行性。

核心发现包括：

Qwen3-Coder-30B 在 MoE 架构和代码微调的双重加持下，成为本次实验的最佳实践模型，能以较低的资源消耗实现较高的修复成功率。

小模型需要更丰富的 RAG 知识库，适当增大 top_k 值（如设为 5）可以有效提升 GPT-OSS-20B 这类模型的性能。

并非所有小模型都适用，Deepseek-R1-8B 等模型的推理稳定性问题是当前的主要障碍。

未来可以进一步探索的方向有：

探索更极致的轻量化：测试 8B 以下模型（如 Qwen2.5-Coder-7B）的可行性。

动态 Top-k 策略：实现一个自适应机制，根据模型置信度或检索结果的相似度分数动态调整 top_k 值。

自动化数据增强：利用 LLM 自身生成合成修复样本，减少人工标注成本。

集成持续学习：使模型能从每次成功的修复中学习，逐步提升自身能力。

优化上下文管理：开发更高效的上下文压缩算法，以处理超长文件的多缺陷修复场景。

参考文献:

- [1] Lin Z, Zhou M, Ma W, Chen C, Yang Y, Wang J, Hu C, Li L. HapRepair: Learn to Repair OpenHarmony Apps[C] // Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (FSE 2025 Industry Track). New York: ACM, 2025: 319-330. DOI: 10.1145/3696630.3728556
- [2] Xia C S, Wei Y, Zhang L. Automated Program Repair in the Era of Large Pre-trained Language Models[C] // Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE 2023). Melbourne: IEEE, 2023: 1482-1494.
- [3] Fan Z, Gao X, Mirchev M, Roychoudhury A, Tan S H. Automated Repair of Programs from Large Language Models[C] // Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE 2023). Melbourne: IEEE, 2023: 1469-1481. DOI: 10.1109/ICSE48619.2023.00128
- [4] Huang K, Meng X, Zhang J, Liu Y, Wang W, Li S, Zhang Y. An Empirical Study on Fine-Tuning Large Language Models of Code for Automated Program Repair[C] // Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE 2023). New York: ACM, 2023: 1162-1174.
- [5] Chen Y, Wu J, Ling X, Li C, Rui Z, Luo T, Wu Y. When Large Language Models Confront Repository-Level Automatic Program Repair: How Well They Done?[J] arXiv preprint arXiv:2403.00448, 2024.
- [6] Zhang J, Chen S, Zhao Y, Li Z. ReCode: Improving LLM-based Code Repair with Fine-Grained Retrieval-Augmented Generation[C] // Proceedings of the 34th ACM International Conference on Information and Knowledge Management (CIKM 2025). arXiv:2509.02330, 2025.
- [7] Mansur E, Chen J, Raza M A, Wardat M. RAGFix: Enhancing LLM Code Repair Using RAG and Stack Overflow Posts[C] // Proceedings of 2024 IEEE International Conference on Big

Data (IEEE BigData 2024). Washington DC: IEEE, 2024. DOI:
10.1109/BigData62323.2024.10825785

[8] Abdinabiev A S, Jung E, Lee B. Enhancing LLM-driven Program Repair through RAG and Test-Method Mapping[J]. Transactions of the Korea Information Processing Society, 2026, 15(1): 61-69. DOI: 10.3745/TKIPS.2026.15.1.61

[9] Guo H, Xie X, Dai H N, Di P, Zhang Y, Tao B, Zheng Z. Accelerating Automatic Program Repair with Dual Retrieval-Augmented Fine-Tuning and Patch Generation on Large Language Models[J] arXiv preprint arXiv:2507.10103, 2025.

[10] Qwen Team. Qwen3-Coder-Next: A MoE Model for Programming Agents[R]. Alibaba Tongyi Qianwen Technical Report, 2026. (模型主页:
<https://www.modelscope.cn/collections/Qwen/Qwen3-Coder-Next>)

[11] Cerebras. Qwen3-Coder-REAP-25B-A3B: Router-weighted Expert Activation Pruning for MoE Compression[R]. Modelscope Technical Report, 2025.
(<https://www.modelscope.cn/models/cerebras/Qwen3-Coder-REAP-25B-A3B>)

[12] Monperrus M. Automatic software repair: A bibliography[J]. ACM Computing Surveys, 2018, 51(1): 17:1-17:24.